

Version 1.1

January 2026



DATA STRUCTURES

MODULE 6 - GRAPH

Author:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

AHMAD NUR MU'MININ

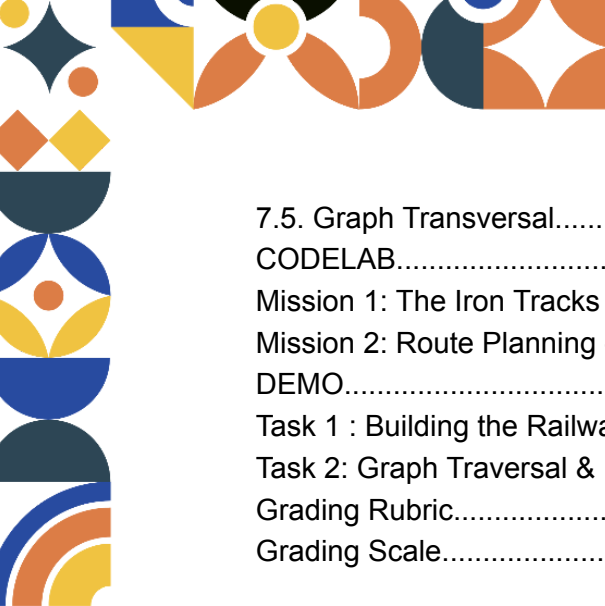
WAHYU ANDIKA

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	4
OBJECTIVES.....	4
MODULE TARGETS.....	4
PREPARATION.....	4
KEYWORDS.....	4
CONTENTS.....	5
1. Basic Concepts of Graphs.....	5
1.1. Definition of a Graph.....	5
1.2. Graph Components: Vertex and Edge.....	5
1.3. Graph Notation: $G = (V, E)$	6
1.4. Characteristics of Relationships in a Graph.....	6
2. Graph Classification.....	7
2.1. Undirected Graph.....	7
2.2. Directed Graph.....	8
2.3. Weighted Graph.....	8
2.4. Conceptual Differences Between Types of Graphs.....	9
3. Fundamental Terms in Graphs.....	10
3.1. Degree and Total Degree.....	10
3.2. In-Degree and Out-Degree.....	10
3.3. Path, Cycle, and Connectivity.....	10
3.4. Impact of Graph Terms on Traversal.....	11
4. Graph Representation.....	11
4.1. Visual Representation of a Graph.....	11
4.2. Adjacency Matrix.....	12
4.3. Adjacency List.....	15
Directed Graph Adjacency List.....	16
5. Graph Traversal.....	16
5.1. Concept and Purpose of Traversal.....	16
5.2. Breadth First Search (BFS).....	17
5.3. Depth First Search (DFS).....	18
6. Design of Traversal Algorithms.....	19
6.1. Logic Flow of BFS.....	19
6.2. Logic Flow of DFS.....	20
7. Implementation Graph.....	23
7.1. Basic Graph.....	23
7.2. Graph Classification.....	24
7.3. Fundamental term.....	27
7.4. Graph Representation.....	29



7.5. Graph Transversal.....	31
CODELAB.....	34
Mission 1: The Iron Tracks - Graph Implementation.....	34
Mission 2: Route Planning - BFS & DFS.....	34
DEMO.....	36
Task 1 : Building the Railway Graph (Adjacency List).....	36
Task 2: Graph Traversal & Pathfinding.....	37
Grading Rubric.....	38
Grading Scale.....	38



INTRODUCTION

OBJECTIVES

1. **Master Graph Fundamentals:** Understand the formal $G = (V, E)$ notation and the structural differences between Undirected, Directed, and Weighted graphs.
2. **Define Connectivity and Metrics:** Identify key properties including Vertex Degree (In/Out-Degree), Paths, Cycles, and the concepts of connected vs. disjoint networks.
3. **Evaluate Representation Techniques:** Compare the implementation and efficiency of Adjacency Matrices versus Adjacency Lists for storing relational data.
4. **Design Traversal Logic:** Formulate the step-by-step logic for Breadth-First Search (BFS) and Depth-First Search (DFS) using Queues, Stacks, and Recursion.
5. **Implement Practical Solutions:** Develop a functional graph-based program capable of representing real-world networks and executing systematic search algorithms.

MODULE TARGETS

1. **Understand and Classify:** Grasp the core components of graphs and distinguish between their various types and relational characteristics.
2. **Represent and Store:** Master the ability to translate visual graph diagrams into programmatic Adjacency Matrices or Adjacency Lists.
3. **Traverse and Analyze:** Execute BFS and DFS algorithms to navigate complex nodes and solve connectivity or pathfinding problems.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Strong motivation and commitment to learning

KEYWORDS

GRAPH, BFS, DFS, MATRIX



CONTENTS

1. Basic Concepts of Graphs

1.1. Definition of a Graph

A graph is a non-linear data structure that represents a set of vertices connected by edges and is formally written as $G = (V, E)$, where V is the set of vertices and E is the set of edges. Vertices represent objects such as cities or atoms and may store labels or values, while edges represent connections between pairs of vertices and may be weighted or unweighted. Graphs support many-to-many relationships, allowing each vertex to connect to multiple other vertices without a fixed hierarchy. Edges may be undirected, where connections are mutual and order does not matter, or directed, where connections have a specific direction. Unlike linear structures and trees, graphs do not have a fixed sequence and may contain cycles, making them suitable for modeling complex relationships. Graphs are commonly implemented using adjacency matrices or adjacency lists to support efficient processing.

1.2. Graph Components: Vertex and Edge

Graph components define how a graph is formed and how relationships between vertices are structured. These components describe the local connectivity of vertices, the types and properties of edges, and the variations that affect traversal and analysis, providing a clear framework for understanding how a graph behaves in both representation and algorithmic processing.

1. Vertices (Nodes)

Each vertex has a local structure defined by its **degree** and **adjacency list**. The degree of a vertex is the number of edges connected to it, which reflects the number of its direct neighbors. An adjacency list records all neighboring vertices and supports efficient local traversal and degree calculation.

2. Edges

Edges can be **directed** or **undirected**. Directed edges have a source and a destination, while undirected edges connect vertices symmetrically. Two-way connections can be modeled using two opposite directed edges. In directed graphs, vertex degree is separated into **in-degree** and **out-degree**, which affects traversal behavior.

3. Edge Attributes

Edges may include numeric attributes such as **weight** and **capacity**. Weights represent quantitative values like distance, cost, or time, while capacity limits the maximum flow through an edge. These attributes add meaning to connections without changing the graph's basic structure.



4. Parallel Edges and Loops

Parallel edges occur when multiple edges connect the same pair of vertices. A **loop** is an edge that connects a vertex to itself and increases the degree of that vertex by two in an undirected graph. These variations appear in multigraphs or pseudographs and influence degree calculation and path evaluation.

5. Topological Implications

The combination of degrees, directions, attributes, and edge variations produces different graph properties. Vertices with high degree indicate strong local connectivity, while parallel edges and bidirectional links provide alternative routes. These properties influence how graph algorithms perform traversal and path analysis.

1.3. Graph Notation: $G = (V, E)$

Formally, a graph is defined as an ordered pair $G = (V, E)$ within a mathematical framework. The set V represents all vertices in the graph, while the set E represents all edges that connect those vertices. This notation clearly separates vertices from their relationships, allowing graph analysis to focus on structure rather than storage details in computer memory. The set V contains unique vertices and is treated as an unordered set, so the order of vertices has no meaning, while the size of V , written as $|V|$, is an important parameter for measuring the scale and complexity of graph algorithms. The set E contains edges expressed as pairs of vertices and is a subset of $V \times V$, representing possible connections between vertices. In directed graphs, each edge is written as an ordered pair (u, v) to preserve direction, whereas in undirected graphs, each edge is written as an unordered pair $\{u, v\}$, meaning the connection is mutual. This formal notation guides graph representations such as adjacency lists and adjacency matrices, where vertices and edges are mapped explicitly, and it supports algorithm analysis by using $|V|$ and $|E|$ as the main factors for evaluating efficiency.

1.4. Characteristics of Relationships in a Graph

Graph relationships describe how vertices are connected and how paths can be formed within a graph. These properties determine the structure of the graph and influence how traversal and analysis algorithms work.

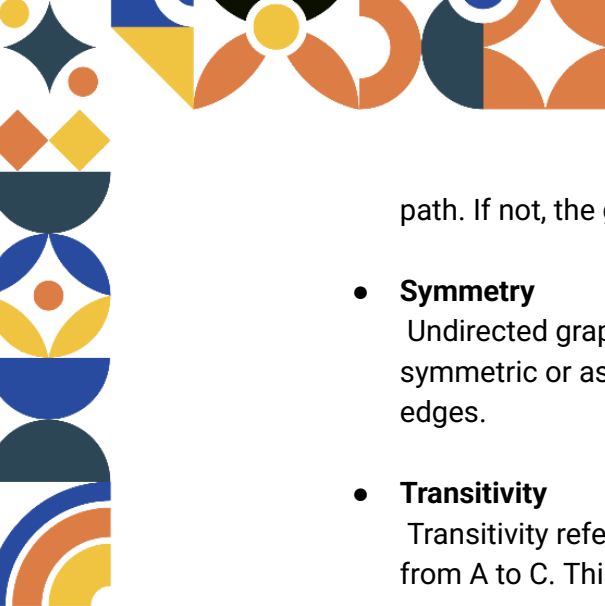
- **Direction of Relationships**

Relationships can be **directed** or **undirected**. Directed relationships are one-way and require ordered vertex pairs, while undirected relationships are mutual and symmetric.

- **Connectivity**

A graph is **connected** if every vertex can reach every other vertex through some





path. If not, the graph is **disconnected** and consists of separate components.

- **Symmetry**
Undirected graphs are symmetric by default, while directed graphs may have symmetric or asymmetric relationships depending on the presence of reverse edges.
- **Transitivity**
Transitivity refers to whether a path from A to B and from B to C implies a path from A to C. This property depends on the existence of explicit or indirect paths.
- **Structural Patterns**
As graphs grow, relationships can form patterns such as cycles or hierarchies. Recognizing these patterns helps in choosing appropriate graph representations and algorithms.

2. Graph Classification

2.1. Undirected Graph

An **undirected graph** is a graph structure in which edges have no direction. Each edge is represented as an unordered pair $\{u, v\}$, meaning the relationship between vertices u and v is symmetric. If u is connected to v , then v is also connected to u . This structure models mutual connections and does not depend on traversal direction.

1. Edge Representation

Edges are unordered pairs $\{u, v\}$, so there is no distinction between (u, v) and (v, u) .

2. Symmetric Connectivity

Connections are two-way by definition, and reverse paths always exist when a path is present.

3. Maximum Number of Edges

With n vertices, an undirected graph can have at most $n(n - 1)/2$ edges.

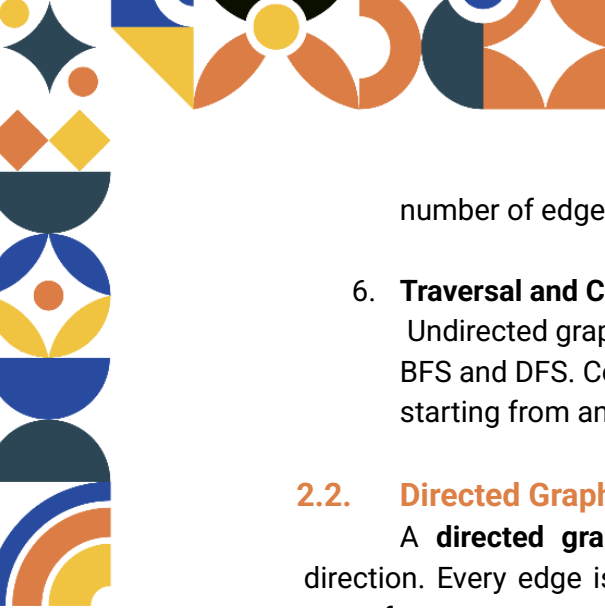
4. Loops and Parallel Edges

A loop connects a vertex to itself and increases its degree by two. The presence of loops or parallel edges forms an undirected multigraph, while simple graphs exclude both.

5. Vertex Degree

The degree of a vertex is the number of incident edges. Each edge contributes one degree to each of its endpoints, and the average degree equals twice the





number of edges divided by the number of vertices.

6. **Traversal and Connectivity**

Undirected graphs allow straightforward traversal using algorithms such as BFS and DFS. Connected components can be identified with a single traversal starting from any vertex.

2.2. **Directed Graph**

A **directed graph** is a graph structure in which each edge has a specific direction. Every edge is represented as an ordered pair $\langle u, v \rangle$, meaning the connection goes from vertex u to vertex v . This direction creates one-way relationships, so a connection from u to v does not imply a connection from v to u .

1. **Edge Representation**

Edges are ordered pairs $\langle u, v \rangle$, where the order indicates direction from the source vertex to the destination vertex.

2. **Asymmetric Connectivity**

Relationships are not necessarily mutual. A reverse path exists only if there is a separate edge $\langle v, u \rangle$.

3. **In-Degree and Out-Degree**

Each vertex has an **in-degree**, which counts incoming edges, and an **out-degree**, which counts outgoing edges. These values describe the role of a vertex within the graph.

4. **Maximum Number of Edges**

With n vertices, a directed graph can have up to $n(n - 1)$ edges, excluding loops.

5. **Traversal Characteristics**

Traversal must follow edge directions, so reachability depends on orientation. Algorithms such as BFS and DFS consider direction when exploring paths.

6. **Typical Applications**

Directed graphs are suitable for modeling one-way relationships such as road systems with one-way streets, task dependencies, and data flow structures.

2.3. **Weighted Graph**

A **weighted graph** is a graph in which each edge is assigned a numeric value called a **weight**. The weight represents a quantitative property of the connection, such as distance, cost, time, or capacity. This value does not change the connectivity structure of the graph but adds meaning to each edge.



1. Edge Weights

Each edge has an associated number that represents the strength or cost of the connection between two vertices.

2. Directed or Undirected

Weighted graphs can be either directed or undirected, depending on whether the edges have direction.

3. Path Evaluation

Paths are evaluated based on the total weight, which is the sum of the weights of all edges along the path.

4. Algorithmic Impact

Weights influence algorithms such as shortest-path and minimum spanning tree, where decisions depend on comparing edge values.

5. Common Applications

Weighted graphs are used in routing, transportation networks, cost optimization, and scheduling problems where numeric evaluation of paths is required.

2.4. Conceptual Differences Between Types of Graphs

Graph Type	Edge Representation	Direction	Key Properties	Degree Concept
Undirected Graph	Unordered pair $\{u, v\}$	No direction	Symmetric connections, mutual reachability	Single degree per vertex
Directed Graph	Ordered pair $\langle u, v \rangle$	One-way	Asymmetric connections, direction matters	In-degree and out-degree
Weighted Graph	Edge with numeric weight	With or without	Quantitative values on edges	Depends on direction (if any)



3. Fundamental Terms in Graphs

3.1. Degree and Total Degree

The **degree of a vertex** is the number of edges directly connected to that vertex. In an undirected graph, each edge increases the degree of both connected vertices by one, and a vertex with degree zero is called an isolated vertex. Degree values describe local connectivity and help identify structurally important vertices, where high-degree vertices often act as hubs or central connectors within the graph.

In directed graphs, degree is divided into **in-degree**, which counts incoming edges, and **out-degree**, which counts outgoing edges. The sum of all degrees is directly related to the number of edges: in undirected graphs it equals twice the number of edges, while in directed graphs total in-degree and total out-degree are equal to the number of edges. Degree analysis is essential in network modeling to study information flow, connectivity patterns, and the functional roles of vertices.

3.2. In-Degree and Out-Degree

The **in-degree** of a vertex is the number of edges directed toward that vertex in a directed graph, while the **out-degree** is the number of edges that originate from it. These two measures are independent and may differ for each vertex, making them essential for analyzing asymmetric relationships. A vertex with high in-degree often acts as a main receiver of connections, while a vertex with high out-degree functions as an active source that distributes connections to others. Vertices with in-degree zero indicate starting points, and vertices with out-degree zero represent endpoints within directed structures.

The total in-degree of all vertices in a directed graph is always equal to the total out-degree, since each edge contributes one unit to both measures. This balance is useful for validating graph data and ensuring structural consistency. In algorithmic analysis, in-degree and out-degree are used to select starting vertices, detect cycles, and perform topological ordering, such as in Kahn's Algorithm. They are also applied in influence and ranking algorithms, including PageRank, as well as in practical domains like routing, flow optimization, and dominance analysis in directed systems.

3.3. Path, Cycle, and Connectivity

A **path** in a graph is an ordered sequence of vertices where each pair of consecutive vertices is connected. A path may be **directed** or **undirected**, depending on the graph type, and its length is measured by the number of edges it contains. Paths are fundamental to graph algorithms used for routing, scheduling, and connectivity analysis.

- **Simple Path:** a path that does not repeat any vertex and does not contain loops.
- **Closed Path:** a path that starts and ends at the same vertex, but may repeat other vertices.



- **Cycle:** a simple closed path where only the starting and ending vertex is repeated.
- **Shortest and Longest Path:** paths evaluated based on minimum or maximum total weight or number of steps.

In directed graphs, a valid path must follow the direction of edges, while in undirected graphs vertices can be traversed in either direction. Cycles play an important role in analysis, since negative cycles affect shortest-path algorithms and cycles are used to detect deadlocks and feasibility issues. Graphs without cycles are called **acyclic graphs**, which form the basis of trees and directed acyclic graphs (DAGs). Understanding paths and cycles helps identify valid traversals and supports efficient graph-based problem solving.

3.4. Impact of Graph Terms on Traversal

Path representation is used to explicitly describe the sequence of connections between vertices in a graph. A path can be written as a sequence of vertices, such as $v_1 \rightarrow v_2 \rightarrow v_3$ in a directed graph or $v_1-v_2-v_3$ in an undirected graph, which helps trace relationships and verify reachability. In implementation, paths may also be stored using indices or data structures such as arrays or linked lists to support traversal and analysis.

Connectivity describes whether vertices in a graph can reach one another through one or more paths. A graph is connected if a path exists between every pair of vertices, while directed graphs distinguish between strong connectivity and weak connectivity based on whether edge directions are considered. Connectivity is analyzed using traversal algorithms such as DFS or BFS to identify connected components, and it plays an important role in network optimization, data flow analysis, and the detection of critical vertices or links.

4. Graph Representation

4.1. Visual Representation of a Graph

A visual representation of a graph shows the graph as a picture to make relationships between nodes easy to see. Nodes are drawn as points or circles, and edges are drawn as lines or arrows. Directed graphs use arrows to show direction, while undirected graphs use simple lines. This helps users understand the graph structure more easily.

The position of nodes affects how clear the graph looks. In graphs with many connections, poor placement can cause lines to overlap. To fix this, different layout methods are used to arrange nodes more clearly. A good layout makes the graph easier to read.

Visual graphs help people quickly see patterns such as groups of nodes, important nodes, paths, and cycles. However, when a graph is too large, it becomes



hard to understand. Because of this, visual representation is best used for small or medium-sized graphs.

Although algorithms are not processed visually by algorithms, visual representation is important for learning, planning, and explaining graph ideas. Understanding visual graphs is an important step before learning graph data structures and algorithms.

4.2. Adjacency Matrix

An adjacency matrix is a graph representation in the form of a square matrix of size $n \times n$, where n is the number of nodes. Each row and column represent a node, and the value at position (i, j) shows whether there is an edge from Vertex i to Vertex j . In an undirected graph, the matrix is symmetric because the connection works both ways. In a directed graph, the value of (i, j) can be different from (j, i) because direction matters.

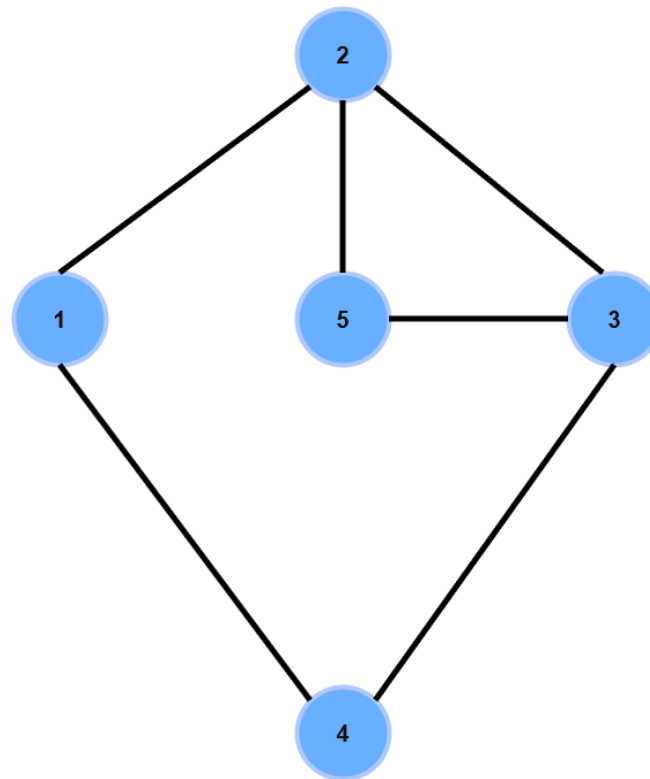
The matrix usually uses binary values: 1 means there is an edge, and 0 means there is no connection. In weighted graphs, the matrix stores edge weights instead of binary values. If there is no edge, the value can be 0 or infinity. This format allows algorithms such as shortest path search to work easily.

The main advantage of an adjacency matrix is fast access. Checking whether two nodes are connected takes constant time. This representation is suitable for dense graphs where many nodes are connected. However, for sparse graphs, it wastes memory because many matrix cells contain zero.

Adjacency matrices are often used in graph algorithms that rely on matrix operations, such as the Floyd–Warshall algorithm. They are also easy to store and implement in programs because of their fixed structure. Understanding adjacency matrices helps students connect graph theory with programming logic.

The provided image represents an **undirected graph** consisting of five vertices (nodes) and six edges (connections).

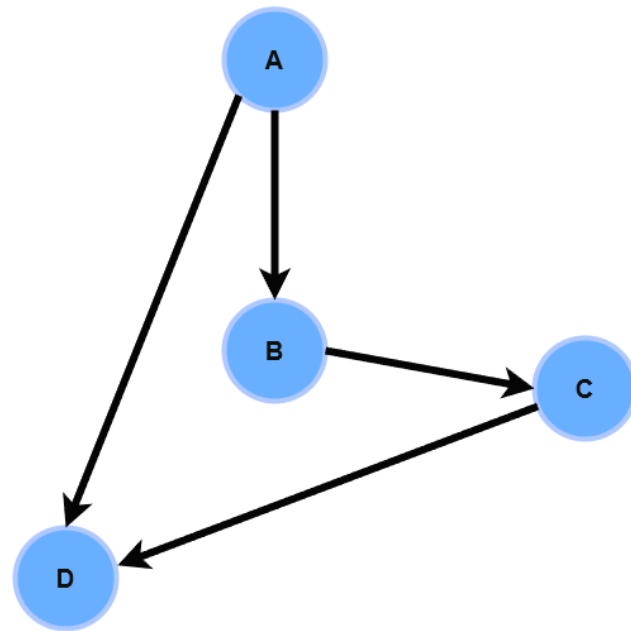




The adjacency matrix for this specific undirected graph is a 5 x 5 symmetric matrix. Each row and column represents a Vertex (1 through 5), where a 1 signifies an existing edge and a 0 signifies no direct connection.

		COLUMN = DESTINATION				
ROW = SOURCE		Vertex 1	Vertex 2	Vertex 3	Vertex 4	Vertex 5
	Vertex 1	0	1	0	1	0
	Vertex 2	1	0	1	0	1
	Vertex 3	0	1	0	1	1
	Vertex 4	1	0	1	0	0
	Vertex 5	0	1	1	0	0





In this case, you have provided a **Directed Graph (Digraph)**. Unlike the previous undirected graph, the connections here have a specific direction indicated by arrows.

When constructing an Adjacency Matrix for a directed graph with **Rows as the Source** and **Columns as the Destination**, a "1" is only placed if an arrow points from the row Vertex to the column node.

	COLUMN = DESTINATION				
ROW = SOURCE		Vertex A	Vertex B	Vertex C	Vertex D
	Vertex A	0	1	0	1
	Vertex B	0	0	1	0
	Vertex C	0	0	0	1
	Vertex D	0	0	0	0



4.3. Adjacency List

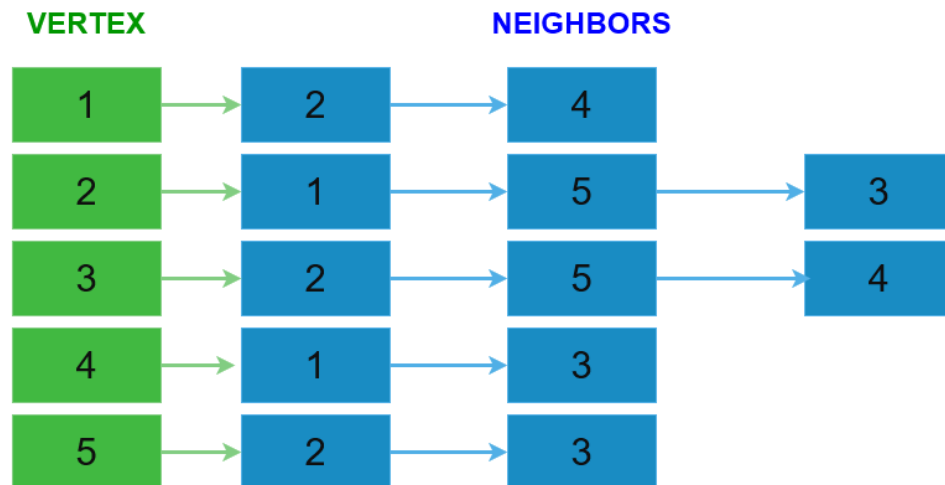
An adjacency list is a graph representation that stores connections as separate lists for each node. Each Vertex has a list of its neighboring nodes that are directly connected by an edge. In a directed graph, the list shows destination nodes based on edge direction. In an undirected graph, each connection appears in both nodes' lists.

This representation is memory efficient for sparse graphs because it only stores existing edges. Unlike an adjacency matrix, it does not waste space for nodes that are not connected. Checking whether two nodes are connected depends on the length of the list, which works well when each Vertex has few edges.

Adjacency lists are usually implemented using arrays or lists. In weighted graphs, each neighbor is stored together with its edge weight. This makes adjacency lists useful for algorithms such as shortest path search.

The main advantage of an adjacency list is flexibility for large graphs. Edges can be added or removed easily, and it works well with BFS and DFS. Because of this, adjacency lists are widely used in real-world graphs like social networks, web graphs, and transportation systems.

Undirected Graph Adjacency List

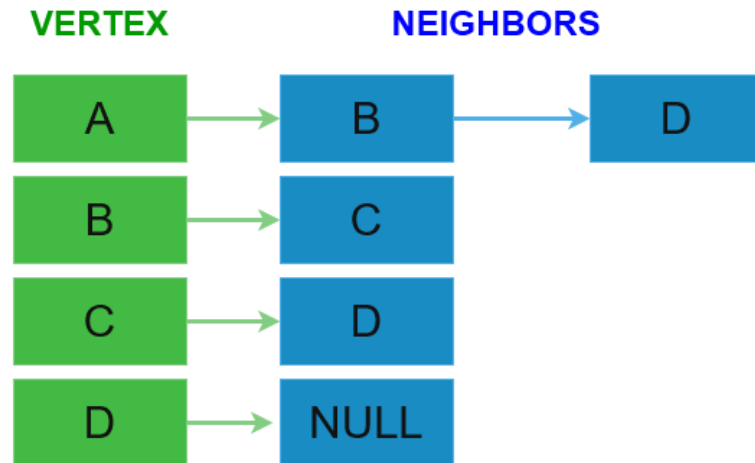


- **Structure Overview:** This visualization represents an adjacency list for an undirected graph, where each connection between two nodes appears in both nodes' respective lists.
- **Vertex and Neighbors:** Similar to the directed version, the green boxes indicate the **Vertices**, and the blue boxes indicate their connected **Neighbors**.
- **Bidirectional Mapping:** Because the graph is undirected, the connections are reciprocal; for example, Vertex 1 lists 2 and 4 as neighbors, and conversely, Vertices 2 and 4 would list 1 in their own lists.



- **Optimization:** This representation avoids wasting space on non-existent connections, making it ideal for large-scale systems like social networks or transportation maps.

Directed Graph Adjacency List



- **Structure Overview:** The image illustrates an adjacency list for a directed graph, which serves as a memory-efficient method for storing connections by maintaining separate lists for each individual node.
- **Vertex and Neighbors:** The green boxes represent the **Vertices** as starting points, while the blue boxes represent the **Neighbors** or destination nodes directly connected by an edge.
- **Directional Logic:** Following the logic of a directed graph, the list shows that Vertex A points to B and D, whereas Vertex D points to **NULL** because it has no outgoing connections.
- **Efficiency:** This structure is highly efficient for sparse graphs as it only stores existing edges and allows for the easy addition or removal of connections.

5. Graph Traversal

5.1. Concept and Purpose of Traversal

Graph traversal is a systematic process of visiting nodes in a graph based on certain rules. The main goal is to explore the graph structure to access data, check connectivity, or create a visit order. Traversal starts from one Vertex and moves to other connected nodes through edges. It is the basis of many graph algorithms such as path finding, connected component detection, and topological sorting.

Traversal is used to check whether a Vertex can be reached from another node. It also helps analyze relationships and levels of connection in a graph. During traversal, the algorithm keeps track of visited nodes and the order of visits to avoid repetition.



There are two main traversal methods: Breadth-First Search (BFS) and Depth-First Search (DFS). BFS visits all neighboring nodes first before moving to the next level. DFS follows one path as deep as possible before going back to explore other paths. These different strategies produce different visit orders.

Traversal algorithms avoid visiting the same Vertex more than once by marking visited nodes. In disconnected graphs, one traversal cannot reach all nodes, so traversal must be repeated from unvisited nodes. Understanding graph traversal is important before learning more advanced graph algorithms.

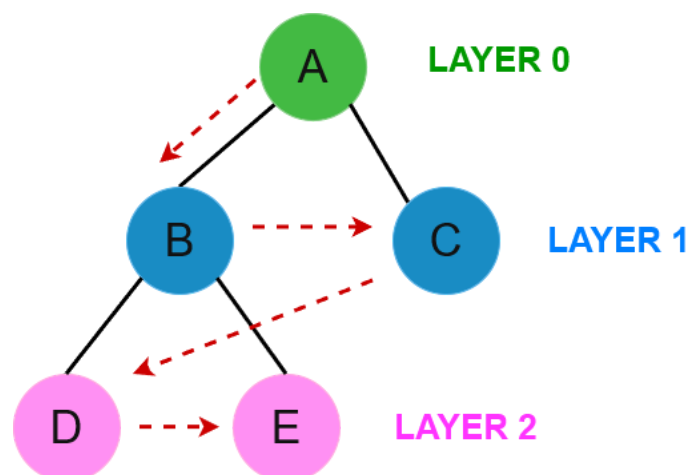
5.2. Breadth First Search (BFS)

Breadth-First Search (BFS) is a graph traversal algorithm that explores nodes based on their distance from the starting node. The process begins at a source Vertex and visits all its neighbors first before moving to the next level. BFS uses a queue to store nodes that will be visited. The first time a Vertex is visited, it is reached through the shortest path in terms of the number of steps.

At the start, all nodes are marked as not visited. A Vertex is marked as visited when it is taken from the queue. When a Vertex is processed, all its unvisited neighbors are added to the queue. This process continues until the queue is empty and all reachable nodes have been visited. This ensures a clear, level-by-level exploration.

The time complexity of BFS is $O(V + E)$, where V is the number of nodes and E is the number of edges. This makes BFS efficient for exploring a whole graph without repeating visits. Extra data structures are used to track visited nodes and visit order.

BFS is commonly used to find the shortest path in unweighted graphs. It is also useful in social networks to find connection levels, in transportation systems to find the minimum number of stops, and to detect connected components or check whether a graph is bipartite.



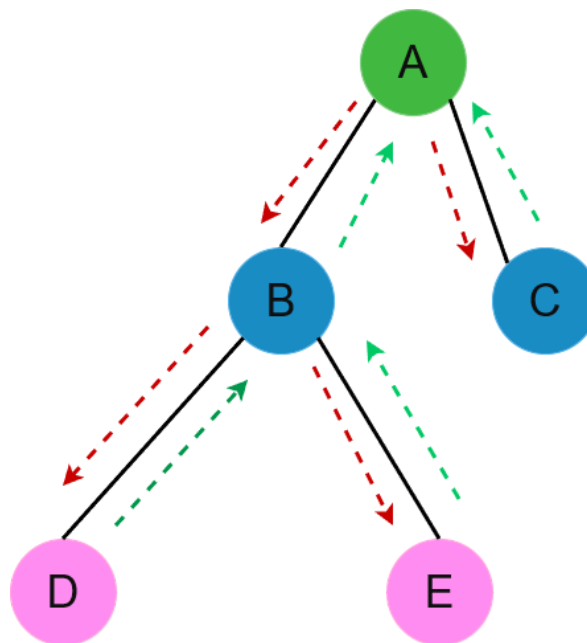
5.3. Depth First Search (DFS)

Depth-First Search (DFS) is a graph traversal algorithm that explores a path as deep as possible before going back to try another path. The process starts from a Vertex and moves to the first unvisited neighbor, continuing deeper until no more nodes can be visited. When it reaches a dead end, the algorithm goes back (backtracks) to explore other branches. This creates a deep, tree-like traversal pattern.

DFS can be implemented using recursion or by using a stack. In the recursive method, the function calls itself for each unvisited neighbor. In the iterative method, nodes are pushed onto a stack and processed from the top. The choice depends on graph depth and system limits.

The time complexity of DFS is $O(V + E)$, the same as BFS, but the visit order is different. DFS is efficient for exploring connected components without repeating visits. Extra memory is used to store visited nodes and manage the stack or recursion.

DFS is widely used to detect cycles, find connected components, and perform topological sorting. It is also used in maze solving, tree evaluation, and decision exploration. Understanding DFS completes the basic knowledge of graph traversal algorithms.



6. Design of Traversal Algorithms

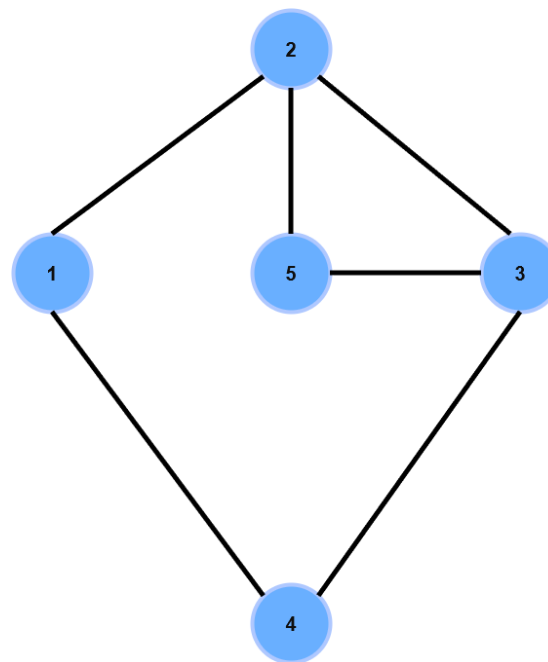
6.1. Logic Flow of BFS

The workflow of Breadth-First Search (BFS) starts by choosing a starting Vertex as the entry point. This Vertex is added to a queue and marked as visited. The algorithm then removes a Vertex from the queue and checks all of its neighbors. Each unvisited neighbor is added to the queue and marked as visited.

The queue ensures that nodes closer to the starting Vertex are visited before nodes that are farther away. This creates a level-by-level exploration based on distance from the start. BFS finishes all nodes at the current level before moving to the next level, which helps find the shortest path in terms of steps.

The main BFS process repeats two actions: taking a Vertex from the queue and exploring its neighbors. Before adding a neighbor, the algorithm checks whether it has already been visited to avoid repetition. This process continues until the queue is empty or all reachable nodes have been visited.

Algorithmically, BFS is implemented using a while loop that runs as long as the queue is not empty. Inside the loop, a Vertex is removed from the queue, its neighbors are checked, and new nodes are added when allowed. Each Vertex is visited only once, so the algorithm is efficient and guaranteed to finish.



Example using the undirected graph, here is how the BFS would execute if we start from **Vertex 1**:



Step	Process	Queue Status (FIFO)	Visited Set	Explanation
Start	-	[1]	{1}	Vertex 1 is the entry point; it is marked as visited and added to the queue.
1	1	[2,4]	{1,2,4}	Vertex 1 is removed. Its neighbors (2, 4) are added to the back of the queue and marked as visited.
2	2	[4,3,5]	{1,2,4,3,5}	Vertex 2 is removed. Neighbors (3, 5) are added to the back. Vertex 4 is now at the front.
3	3	[3,5]	{1,2,4,3,5}	Vertex 4 is removed. Its neighbors (1, 3) are already in the Visited set, so they are not added again.
4	4	[5]	{1,2,4,3,5}	Vertex 3 is removed. All its neighbors are already visited, so the queue shrinks.
5	5	[]	{1,2,4,3,5}	Vertex 5 is removed. The queue is now empty, and the search is complete.

Final Traversal Order: 1 -> 2 -> 4 -> 3 -> 5, Understanding the BFS workflow is important for developing more advanced algorithms, such as tracking levels, storing shortest paths, or extending BFS for more complex graph problems.

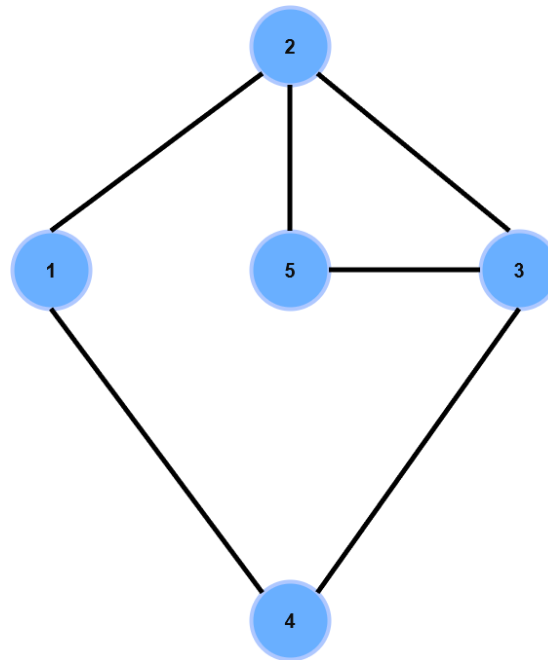
6.2. Logic Flow of DFS

The basic process of Depth-First Search (DFS) starts by choosing a starting node. This Vertex is marked as visited and becomes the starting point of the search. The algorithm then moves to one unvisited neighbor and continues exploring deeper in the same way. When no unvisited neighbor is found, DFS goes back to the previous Vertex to explore another path.



DFS can be implemented using recursion or by using a stack. In the recursive approach, each function call represents the current depth of the search. In the iterative approach, a stack stores nodes that still need to be explored. Both methods give the same result if neighbors are visited in the same order.

When a Vertex is processed, all of its neighbors are checked one by one. Each unvisited neighbor is immediately explored. Visited nodes are not processed again, so the algorithm is efficient and always finishes. DFS follows a Last-In, First-Out (LIFO) rule, which means it explores the deepest path first.



Example using the undirected graph, here is how the DFS would execute if we start from **Vertex 1**:

Step	Process	Stack Status (LIFO)	Visited Set	Explanation
Start	-	[1]	{1}	Vertex 1 is the entry point; it is marked as visited and added to the stack..
1	1	[4,2]	{1,2}	Vertex 1 is removed. Its neighbors (2, 4) are added to the stack. Since 2 is on top (right side), it will be processed next.



2	2	[4,5,3]	{1,2,3}	Vertex 2 is removed. Neighbors (3, 5) are added. 3 is now on top.
3	3	[4,5,4,5]	{1,2,3,5}	Vertex 3 is removed. Neighbors (4, 5) are added. 5 is now the deepest path..
4	4	[4,5,4]	{1,2,3,5}	Vertex 5 is removed. It has no unvisited neighbors, so the algorithm begins backtracking to the next item in the stack.
5	5	[]	{1,2,4,3,5,4}	Vertex 4 is removed from the bottom of the stack. Since it's the last reachable node, the process finishes.

Final Traversal Order: 1 -> 2 -> 3 -> 5 -> 4, DFS is useful for tasks such as detecting cycles, finding connected components, and performing topological sorting. It is also used in maze solving, tree traversal, and hierarchical data processing. Understanding the DFS workflow is important for learning more advanced graph algorithms.



7. Implementation Graph

7.1. Basic Graph

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class GraphComponents {
5
6      // A Vertex represents a node in the graph
7      static class Vertex {
8          String label;
9
10         public Vertex(String label) {
11             this.label = label;
12         }
13
14         @Override
15         public String toString() {
16             return label;
17         }
18     }
19
20     // An Edge represents a connection between two vertices
21     static class Edge {
22         Vertex source;
23         Vertex destination;
24         int weight; // Optional: for weighted graphs
25
26         public Edge(Vertex source, Vertex destination) {
27             this.source = source;
28             this.destination = destination;
29             this.weight = 0; // Default weight
30         }
31
32         // Constructor for weighted edge
33         public Edge(Vertex source, Vertex destination, int weight) {
34             this.source = source;
35             this.destination = destination;
36             this.weight = weight;
37         }
38
39         @Override
40         public String toString() {
41             return source + " -> " + destination + (weight != 0 ? " [" + weight + "]" : "");
42         }
43     }
44
45     public static void main(String[] args) {
46         // Demonstration
47         Vertex v1 = new Vertex("A");
48         Vertex v2 = new Vertex("B");
49
50         Edge edge = new Edge(v1, v2);
51
52         System.out.println("Vertex 1: " + v1);
53         System.out.println("Vertex 2: " + v2);
54         System.out.println("Edge created: " + edge);
55     }
56 }
57

```

This code defines simple graph components using Vertex and Edge classes and demonstrates their usage in the main method. The Vertex class represents a Vertex with a label and overrides toString so the label is printed directly. The Edge class represents a connection between two vertices and supports both unweighted and weighted edges by providing two constructors with a default weight of zero for unweighted edges. It also overrides toString to display the connection from source to destination and includes the weight only when it is not zero. In main two vertices are created labeled A and B then an edge is created between them and the program prints both vertices and the edge showing how these classes model a basic graph structure.



7.2. Graph Classification

```

1  import java.util.*;
2
3  public class UndirectedGraph {
4      private Map<String, List<String>> adjList = new HashMap<>();
5
6      // Add a vertex to the graph
7      public void addVertex(String label) {
8          adjList.putIfAbsent(label, new ArrayList<>());
9      }
10
11     // Add an undirected edge
12     public void addEdge(String src, String dest) {
13         // Add edge from src to dest
14         adjList.get(src).add(dest);
15         // Add edge from dest to src (Because it's undirected)
16         adjList.get(dest).add(src);
17     }
18
19     public void printGraph() {
20         System.out.println("Undirected Graph Adjacency List:");
21         for (String node : adjList.keySet()) {
22             System.out.println(node + " <-> " + adjList.get(node));
23         }
24     }
25
26     public static void main(String[] args) {
27         UndirectedGraph graph = new UndirectedGraph();
28         graph.addVertex("Alice");
29         graph.addVertex("Bob");
30         graph.addVertex("Charlie");
31
32         graph.addEdge("Alice", "Bob");
33         graph.addEdge("Bob", "Charlie");
34
35         // Alice is connected to Bob, and Bob is connected to Alice
36         graph.printGraph();
37     }
38 }

```

This code implements a simple undirected graph using an adjacency list stored in a HashMap where each vertex label maps to a list of its connected neighbors. The `addVertex` method adds a new Vertex to the graph if it does not already exist while `addEdge` creates a two way connection by adding each vertex to the other adjacency list because the graph is undirected. The `printGraph` method displays all vertices along with their connected nodes. In the main method three vertices Alice Bob and Charlie are created then edges are added between Alice and Bob and between Bob and Charlie and finally the adjacency list is printed showing how each Vertex is connected.



```

1  import java.util.*;
2
3  public class DirectedGraph {
4      private Map<String, List<String>> adjList = new HashMap<>();
5
6      public void addVertex(String label) {
7          adjList.putIfAbsent(label, new ArrayList<>());
8      }
9
10     // Add a directed edge
11     public void addEdge(String src, String dest) {
12         // Only add edge from src to dest
13         adjList.get(src).add(dest);
14         // Do NOT add from dest to src
15     }
16
17     public void printGraph() {
18         System.out.println("Directed Graph Adjacency List:");
19         for (String node : adjList.keySet()) {
20             System.out.println(node + " -> " + adjList.get(node));
21         }
22     }
23
24     public static void main(String[] args) {
25         DirectedGraph graph = new DirectedGraph();
26         graph.addVertex("A");
27         graph.addVertex("B");
28         graph.addVertex("C");
29
30         graph.addEdge("A", "B"); // A points to B
31         graph.addEdge("B", "C"); // B points to C
32
33         // C does not point back to B or A automatically
34         graph.printGraph();
35     }
36 }

```

This code implements a directed graph using an adjacency list stored in a HashMap where each vertex maps to a list of outgoing connections. The addVertex method adds a new Vertex if it does not already exist while addEdge creates a one way connection from the source vertex to the destination vertex without adding the reverse link. The printGraph method displays each vertex and the nodes it points to. In the main method three vertices A B and C are created then edges are added from A to B and from B to C and finally the adjacency list is printed showing that connections only flow in one direction.



```

1  import java.util.*;
2
3  public class WeightedGraph {
4
5      // Edge class to store destination and weight
6      static class Edge {
7          String dest;
8          int weight;
9
10         public Edge(String dest, int weight) {
11             this.dest = dest;
12             this.weight = weight;
13         }
14
15         @Override
16         public String toString() {
17             return "(" + dest + ", " + weight + ")";
18         }
19     }
20
21     private Map<String, List<Edge>> adjList = new HashMap<>();
22
23     public void addVertex(String label) {
24         adjList.putIfAbsent(label, new ArrayList<>());
25     }
26
27     // Add a weighted edge (Undirected in this example, but can be directed)
28     public void addEdge(String src, String dest, int weight) {
29         adjList.get(src).add(new Edge(dest, weight));
30         adjList.get(dest).add(new Edge(src, weight));
31     }
32
33     public void printGraph() {
34         System.out.println("Weighted Graph Adjacency List:");
35         for (String node : adjList.keySet()) {
36             System.out.println(node + " -> " + adjList.get(node));
37         }
38     }
39
40     public static void main(String[] args) {
41         WeightedGraph graph = new WeightedGraph();
42         graph.addVertex("New York");
43         graph.addVertex("London");
44         graph.addVertex("Tokyo");
45
46         graph.addEdge("New York", "London", 5500);
47         graph.addEdge("London", "Tokyo", 9500);
48         graph.addEdge("New York", "Tokyo", 10000);
49
50         graph.printGraph();
51     }
52 }

```

This code implements a weighted graph using an adjacency list where each vertex maps to a list of Edge objects that store a destination and a weight. The Edge class represents a connection with its cost and overrides toString so each edge is printed as a pair containing the destination and weight. The addVertex method adds a Vertex if it does not already exist while addEdge creates an undirected weighted connection by adding an Edge to both vertices adjacency lists. The printGraph method displays every vertex along with its connected edges. In main three cities New York, London, and Tokyo are added as vertices then weighted edges are created between them to represent distances and finally the adjacency list is printed to show all connections and their weights.



7.3. Fundamental term

```

1  import java.util.*;
2
3  public class GraphDegree {
4      private Map<String, List<String>> adjList = new HashMap<>();
5
6      public void addEdge(String src, String dest) {
7          adjList.putIfAbsent(src, new ArrayList<>());
8          adjList.putIfAbsent(dest, new ArrayList<>());
9          adjList.get(src).add(dest);
10         adjList.get(dest).add(src);
11     }
12
13     // Get degree of a specific vertex
14     public int getDegree(String vertex) {
15         if (!adjList.containsKey(vertex))
16             return 0;
17         return adjList.get(vertex).size();
18     }
19
20     // Calculate total degree of the graph
21     public int getTotalDegree() {
22         int total = 0;
23         for (String v : adjList.keySet()) {
24             total += getDegree(v);
25         }
26         return total;
27     }
28
29     public static void main(String[] args) {
30         GraphDegree g = new GraphDegree();
31         g.addEdge("A", "B");
32         g.addEdge("B", "C");
33         g.addEdge("A", "C");
34
35         System.out.println("Degree of A: " + g.getDegree("A")); // 2 (to B, to C)
36         System.out.println("Degree of B: " + g.getDegree("B")); // 2
37         System.out.println("Total Degree: " + g.getTotalDegree()); // 6 (2 * 3 edges)
38     }
39 }

```

This code builds an undirected graph using an adjacency list and provides methods to calculate vertex degree and total graph degree. The `addEdge` method ensures both vertices exist and then adds each one to the other neighbor list to represent a two way connection. The `getDegree` method returns how many connections a specific vertex has while `getTotalDegree` sums the degrees of all vertices in the graph. In the main method vertices A, B, and C are connected with three edges and the program prints the degree of A and B as well as the total degree which equals twice the number of edges because each undirected edge contributes to two vertices.



```

1  import java.util.*;
2
3  public class DirectedGraphDegree {
4      private Map<String, List<String>> adjList = new HashMap<>();
5
6      public void addEdge(String src, String dest) {
7          adjList.putIfAbsent(src, new ArrayList<>());
8          adjList.putIfAbsent(dest, new ArrayList<>()); // Ensure dest exists
9          adjList.get(src).add(dest);
10     }
11
12     public int getOutDegree(String vertex) {
13         if (!adjList.containsKey(vertex))
14             return 0;
15         return adjList.get(vertex).size();
16     }
17
18     public int getInDegree(String vertex) {
19         int count = 0;
20         for (String v : adjList.keySet()) {
21             for (String neighbor : adjList.get(v)) {
22                 if (neighbor.equals(vertex)) {
23                     count++;
24                 }
25             }
26         }
27         return count;
28     }
29
30     public static void main(String[] args) {
31         DirectedGraphDegree g = new DirectedGraphDegree();
32         g.addEdge("A", "B"); // A -> B
33         g.addEdge("B", "C"); // B -> C
34         g.addEdge("A", "C"); // A -> C
35
36         System.out.println("Node A - In: " + g.getInDegree("A") + ", Out: " + g.getOutDegree("A")); // In: 0, Out: 2
37         System.out.println("Node B - In: " + g.getInDegree("B") + ", Out: " + g.getOutDegree("B")); // In: 1, Out: 1
38         System.out.println("Node C - In: " + g.getInDegree("C") + ", Out: " + g.getOutDegree("C")); // In: 2, Out: 0
39     }
40 }
41

```

This code implements a directed graph and calculates both in degree and out degree for each vertex. The graph is stored using an adjacency list where `addEdge` creates a one way connection from the source to the destination while also ensuring both vertices exist in the map. The `getOutDegree` method returns how many outgoing edges a vertex has by checking the size of its adjacency list, while `getInDegree` counts how many times a vertex appears in all other adjacency lists to measure incoming edges. In the main method nodes A, B, and C are connected with directed edges and the program prints the in degree and out degree of each Vertex showing how many edges enter and leave every vertex.



7.4. Graph Representation

```

1  public class AdjacencyMatrix {
2      private int[][] matrix;
3      private int numVertices;
4
5      public AdjacencyMatrix(int numVertices) {
6          this.numVertices = numVertices;
7          matrix = new int[numVertices][numVertices];
8      }
9
10     // Add edge (Undirected)
11     public void addEdge(int i, int j) {
12         if (i >= 0 && i < numVertices && j >= 0 && j < numVertices) {
13             matrix[i][j] = 1;
14             matrix[j][i] = 1;
15         }
16     }
17
18     // Remove edge
19     public void removeEdge(int i, int j) {
20         if (i >= 0 && i < numVertices && j >= 0 && j < numVertices) {
21             matrix[i][j] = 0;
22             matrix[j][i] = 0;
23         }
24     }
25
26     public void printMatrix() {
27         System.out.println("Adjacency Matrix:");
28         for (int i = 0; i < numVertices; i++) {
29             for (int j = 0; j < numVertices; j++) {
30                 System.out.print(matrix[i][j] + " ");
31             }
32             System.out.println();
33         }
34     }
35
36     public static void main(String[] args) {
37         AdjacencyMatrix graph = new AdjacencyMatrix(4); // 4 vertices: 0, 1, 2, 3
38
39         graph.addEdge(0, 1);
40         graph.addEdge(0, 2);
41         graph.addEdge(1, 2);
42         graph.addEdge(2, 3);
43
44         graph.printMatrix();
45     }
46 }

```

This code represents an undirected graph using an adjacency matrix where a two dimensional array stores connections between vertices. The constructor initializes the matrix based on the number of vertices. The addEdge method connects two vertices by setting both matrix[i][j] and matrix[j][i] to one while removeEdge removes a connection by setting those values back to zero. The printMatrix method displays the entire matrix row by row. In the main method a graph with four vertices is created, several edges are added and finally the adjacency matrix is printed to show which vertices are connected.



```

1  import java.util.*;
2
3  public class AdjacencyList {
4      private Map<Integer, List<Integer>> adjList;
5
6      public AdjacencyList() {
7          adjList = new HashMap<>();
8      }
9
10     // Add vertex
11     public void addVertex(int v) {
12         adjList.putIfAbsent(v, new ArrayList<>());
13     }
14
15     // Add edge
16     public void addEdge(int src, int dest) {
17         addVertex(src);
18         addVertex(dest);
19         adjList.get(src).add(dest);
20         adjList.get(dest).add(src); // Undirected
21     }
22
23     public void printGraph() {
24         System.out.println("Adjacency List:");
25         for (Integer v : adjList.keySet()) {
26             System.out.print(v + " -> ");
27             for (Integer neighbor : adjList.get(v)) {
28                 System.out.print(neighbor + " ");
29             }
30             System.out.println();
31         }
32     }
33
34     public static void main(String[] args) {
35         AdjacencyList graph = new AdjacencyList();
36
37         graph.addEdge(0, 1);
38         graph.addEdge(0, 4);
39         graph.addEdge(1, 2);
40         graph.addEdge(1, 3);
41         graph.addEdge(1, 4);
42         graph.addEdge(2, 3);
43         graph.addEdge(3, 4);
44
45         graph.printGraph();
46     }
47 }

```

This code implements an undirected graph using an adjacency list stored in a HashMap where each integer vertex maps to a list of its neighboring vertices. The addVertex method ensures a vertex exists before connections are added, while addEdge creates a two way link by adding each vertex to the other neighbor list. The printGraph method displays every vertex followed by all vertices connected to it. In the main method several edges are added between numbered vertices and finally the adjacency list is printed to show the full structure of the graph.



7.5. Graph Transversal

```

1  import java.util.*;
2
3  public class BFS {
4      private int V; // Number of vertices
5      private LinkedList<Integer> adj[]; // Adjacency Lists
6
7      @SuppressWarnings("unchecked")
8      public BFS(int v) {
9          V = v;
10         adj = new LinkedList[V];
11         for (int i = 0; i < v; ++i)
12             adj[i] = new LinkedList();
13     }
14
15     void addEdge(int v, int w) {
16         adj[v].add(w);
17     }
18
19     // BFS traversal from a given source s
20     void runBFS(int s) {
21         // Mark all the vertices as not visited (By default false)
22         boolean visited[] = new boolean[V];
23
24         // Create a queue for BFS
25         LinkedList<Integer> queue = new LinkedList<Integer>();
26
27         // Mark the current node as visited and enqueue it
28         visited[s] = true;
29         queue.add(s);
30
31         System.out.println("Starting BFS from node " + s + ":");
32
33         while (queue.size() != 0) {
34             // Dequeue a vertex from queue and print it
35             s = queue.poll();
36             System.out.print(s + " ");
37
38             // Get all adjacent vertices of the dequeued vertex s
39             // If a adjacent has not been visited, then mark it visited and enqueue it
40             Iterator<Integer> i = adj[s].listIterator();
41             while (i.hasNext()) {
42                 int n = i.next();
43                 if (!visited[n]) {
44                     visited[n] = true;
45                     queue.add(n);
46                 }
47             }
48             System.out.println();
49         }
50     }
51
52     public static void main(String args[]) {
53         BFS g = new BFS(4);
54
55         g.addEdge(0, 1);
56         g.addEdge(0, 2);
57         g.addEdge(1, 2);
58         g.addEdge(2, 0);
59         g.addEdge(2, 3);
60         g.addEdge(3, 3);
61
62         g.runBFS(2);
63     }
64 }

```

This code implements Breadth First Search on a graph using adjacency lists. The BFS class stores the number of vertices and an array of linked lists to represent connections between nodes. The addEdge method adds a directed edge from one vertex to another. The runBFS method starts from a given source Vertex by marking it as visited and placing it in a queue then repeatedly removes a vertex from the queue prints it and visits all its unvisited neighbors by marking them and adding them to the queue. This continues until the queue is empty so nodes are processed level by level. In the main method, a graph with four vertices is created, several edges are added and BFS is executed starting from vertex 2 producing the traversal order.



```

1  import java.util.*;
2
3  public class DFS {
4      private int V;
5      private LinkedList<Integer> adj[];
6
7      @SuppressWarnings("unchecked")
8      public DFS(int v) {
9          V = v;
10         adj = new LinkedList[V];
11         for (int i = 0; i < v; ++i)
12             adj[i] = new LinkedList();
13     }
14
15     void addEdge(int v, int w) {
16         adj[v].add(w);
17     }
18
19     // A function used by DFS
20     void DFSUtil(int v, boolean visited[]) {
21         // Mark the current node as visited and print it
22         visited[v] = true;
23         System.out.print(v + " ");
24
25         // Recur for all the vertices adjacent to this vertex
26         Iterator<Integer> i = adj[v].listIterator();
27         while (i.hasNext()) {
28             int n = i.next();
29             if (!visited[n])
30                 DFSUtil(n, visited);
31         }
32     }
33
34     // The function to do DFS traversal. It uses recursive DFSUtil()
35     void runDFS(int v) {
36         boolean visited[] = new boolean[V];
37         System.out.println("Starting DFS from node " + v + ":");
38         DFSUtil(v, visited);
39         System.out.println();
40     }
41
42     public static void main(String args[]) {
43         DFS g = new DFS(4);
44
45         g.addEdge(0, 1);
46         g.addEdge(0, 2);
47         g.addEdge(1, 2);
48         g.addEdge(2, 0);
49         g.addEdge(2, 3);
50         g.addEdge(3, 3);
51
52         g.runDFS(2);
53     }
54 }

```



A decorative vertical bar on the left side of the page, composed of various geometric shapes in blue, orange, and yellow, including circles, diamonds, and semi-circles.

This code implements Depth First Search on a graph using adjacency lists and recursion. The DFS class stores the number of vertices and an array of linked lists to represent edges, and addEdge creates a directed connection between vertices. The DFSUtil method marks the current Vertex as visited, prints it and then recursively visits each unvisited neighbor, which causes the traversal to go as deep as possible before backtracking. The runDFS method initializes the visited array, starts the traversal from a chosen Vertex and prints a header. In the main method a graph with four vertices is created, several edges are added and DFS is executed starting from vertex 2 producing the depth first traversal order.



CODELAB

Mission 1: The Iron Tracks - Graph Implementation

Objective Help the **Chief Engineer** connect the major cities of Java! You must implement an **Adjacency List** to represent stations and the tracks linking them.

Core Concepts

- **Vertex (Station):** A Vertex in the graph.
- **Edge (Track):** A connection between two nodes.
- **Adjacency List:** A collection where each Vertex maps to a list of its neighbors.

Your Tasks

1. **Station Log (addVertex):**
 - Ensure stations are registered in the system.
 - *Note: This functionality is marked as **already provided** in the starter code.*
2. **Track Laying (addEdge):**
 - Implement the logic to connect two stations.
 - **Requirement:** Since trains go both ways (Undirected Graph), if Station A connects to Station B, then Station B must also connect to Station A.
 - *Hint:* Add the destination to the source's list, and vice versa.

Mission 2: Route Planning - BFS & DFS

Objective with the physical tracks established in Mission 1, the network now requires navigation logic. Your objective is to implement two fundamental graph traversal algorithms: **Breadth-First Search (BFS)** for analyzing immediate connectivity (like a radar scan) and **Depth-First Search (DFS)** for exhaustive route exploration.

Your Tasks

1. **Radar Scan (bfs)** Implement the BFS algorithm using a **Queue** (FIFO) data structure.
 - **Logic:** Start at a designated station, visit all its immediate neighbors, then move to the neighbors' neighbors (level-by-level traversal).
 - **Implementation Details:**
 - Initialize a Queue and add the starting station.
 - Use a Set or boolean array to keep track of `visited` stations to avoid infinite loops.

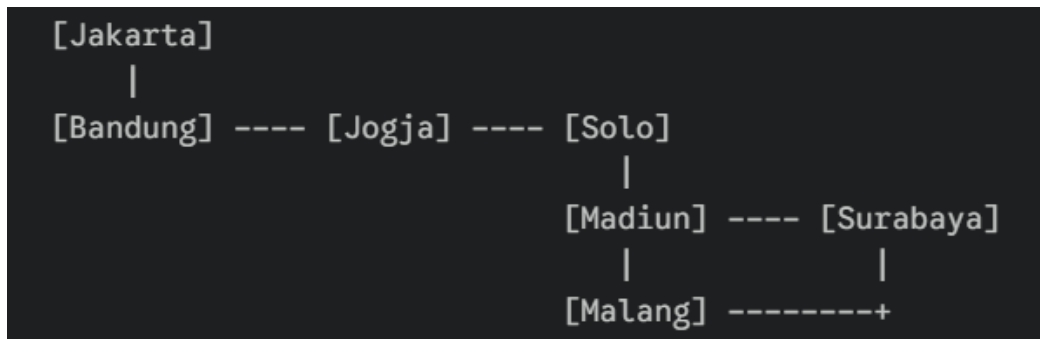


- **Loop:** While the queue is not empty, `poll()` the current station, print/process it, and add its unvisited neighbors to the queue.

2. Deep Exploration (dfs) Implement the DFS algorithm using **Recursion** (utilizing the system stack).

- **Logic:** Plunge as deep as possible along one specific branch of tracks. Continue until you hit a dead end or a previously visited station, then **backtrack** to explore other branches.
- **Implementation Details:**
 - Create a recursive function that takes the current station and the visited set as parameters.
 - Mark the current station as visited and print/process it.
 - **Loop:** Iterate through the neighbors. If a neighbor has not been visited, recursively call the function for that neighbor.

Visual Guide (The Java Railway Map) Use this map to test your traversals. Note the cycle between Madiun, Surabaya, and Malang.



Output:

```

Station Jakarta connects to: [Bandung]
Station Bandung connects to: [Jakarta, Jogja]
Station Solo connects to: [Jogja, Madiun]
Station Madiun connects to: [Solo, Surabaya, Malang]
Station Malang connects to: [Madiun, Surabaya]
Station Jogja connects to: [Bandung, Solo]

[BFS Traversal starting from Surabaya]
Surabaya -> Madiun -> Malang -> Solo -> Jogja -> Bandung -> Jakarta -> END

[DFS Traversal starting from Surabaya]
Surabaya -> Madiun -> Solo -> Jogja -> Bandung -> Jakarta -> Malang -> END
  
```

You can clone the project from this GitHub repository, specifically from the 'modul6' branch
 " `git clone -b modul6 https://github.com/dikameo/DATA-STRUCTURE.git "`



DEMO

Task 1 : Building the Railway Graph (Adjacency List)

Objective

Create the core graph structure where railway stations represent Nodes (Vertices) and tracks represent Edges connecting them. The edges must be weighted to include multiple types of costs: distance, travel time, and fare information. An Adjacency List implementation is required for efficient space utilization.

Detailed Requirements

1. Class `Station` (Vertex)

Create a class to represent a Vertex in the graph.

- Attributes:
 - `code` (String): A unique key identifier for the station (e.g., "NY", "LON").
 - `name` (String): The full name of the station.
 - `city` (String): The city where the station is located.

2. Class `Route` (Edge)

Create a class to represent a connection between two stations.

- Attributes:
 - `destination` (String): The station code of the connected station.
 - `distance` (int): Weight 1 - The physical distance between stations.
 - `travelTime` (int): Weight 2 - The time taken to travel this route.
 - `fare` (double): Weight 3 - The cost of the ticket for this segment.

3. Class `RailwayGraph`

Implement the graph logic using an Adjacency List.

- Data Structure:
 - `HashMap<String, Station>`: To store vertex details mapped by their station code.
 - `HashMap<String, ArrayList<Route>>`: The actual adjacency list storing connections for each station.
- Required Operations:
 - Add Station: Insert a new vertex (station) into the graph.
 - Add Route: Insert a weighted edge between two stations. Note: Since railway tracks usually allow travel in both directions, implement this as an undirected graph (add edges A -> B and B -> A).
 - Get Connections: Retrieve all routes originating from a specific station.
 - Remove: Implement logic to remove a station (which must also remove all associated edges) or remove a specific route between two stations.

Task 2: Graph Traversal & Pathfinding

Objective Implement standard graph traversal algorithms to navigate the railway network. These algorithms will be used to visualize network connectivity and calculate paths based on specific criteria (e.g., fewest stops).

Detailed Requirements

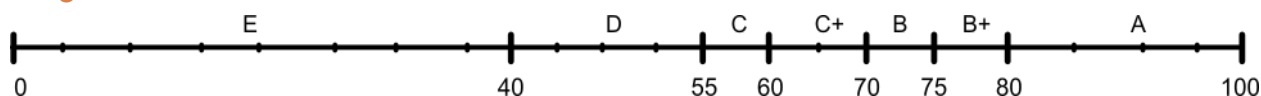
1. Breadth-First Search (BFS) implements the BFS algorithm using a Queue (FIFO) data structure.
 - Purpose: To explore neighbor stations level-by-level (layer-by-layer) starting from a source station.
 - Required Applications:
 - `bfsTraversal(String startCode)`: A method to print all reachable stations in the exact order they are visited by the BFS algorithm.
 - `findPathBFS(String start, String end)`: A method to find the route between two stations that has the minimum number of stops. (Note: In an unweighted context, BFS guarantees the shortest path in terms of the number of edges/stops).
2. Depth-First Search (DFS) implements the DFS algorithm using a Stack (LIFO) or Recursion.
 - Purpose: To explore as deep as possible along each branch of tracks before backtracking.
 - Required Applications:
 - `dfsTraversal(String startCode)`: A method to print stations to demonstrate the exploration order. This is theoretically useful for tasks like detecting cycles in the railway network or analyzing deep connectivity.



Grading Rubric

Assessment Aspect	Points (Total 100%)
Codelab	Total 10%
The program successfully executes	10%
Demo	Total 90%
Task 1	45%
Implement Graph	10%
Running Smoothly	10%
Answer Questions	25%
Task 2	45%
Implement BFS & DFS	10%
Running Smoothly	10%
Answer Questions	25%

Grading Scale



- A** = (81 - 100) → **Sepuh**
- B+** = (75 - 80) → **Very Good**
- B** = (70 - 74) → **Good**
- C+** = (60 - 69) → **Fairly Good**
- C** = (55 - 59) → **Fair**
- D** = (41 - 54) → **Poor**
- E** = (0 - 40) → **Bro really...**

